

Hotel Booking & Management System

A Normalised Relational Design with a Database-Driven Pricing Agent

EBU5503 Database Systems — Group Coursework Report (2025/26)

Scenario 2: Hotel Reservation & Management | **Group 13** | 20 May 2026

Xinke Lyu241119389

Hao Wang241119563

Yiju Huang241119116

Haoliang Fan241119035

Zelin Li241119161

Xinyu Li241119183

Xinzhong Liu241119312

1. Detailed Requirements and Assumptions

1.1 Scope and basic requirements

The system records the four data domains mandated by Scenario 2 – **guests**, **rooms**, **bookings/reservations** and **payments** – together with the operational artefacts a non-trivial Property Management System inevitably accretes around them. A booking belongs to exactly one guest, may span multiple rooms over arbitrary date ranges, and is settled by one or more payment transactions including deposits, settlements and refunds.

1.2 Additional requirements (creative extensions)

Five extensions are introduced, each justified by a concrete operational need rather than an academic flourish.

Extension	Justification and real-world mapping
Loyalty Program	Tiered membership (Bronze/Silver/Gold/Platinum) mirrors Marriott Bonvoy and Hilton Honors; supports retention analytics (Q4, Q11).
Dynamic Pricing Rule	Event- and season-based multipliers (Christmas, Spring Festival, Summer, weekdays) move the design from static rate cards to revenue-managed pricing.
Room Type abstraction	Separates physical room from pricing category, removing the update anomaly that would arise from storing rate per row of ROOM (2NF).
Department & Staff	Models internal organisation, prerequisite for any task-assignment analytics.
Task Log	Records cleaning, maintenance and inspection per (Staff, Room, Task_Type); a true ternary relationship that underpins Q6/Q8.

1.3 Key assumptions

- A booking can span multiple rooms with independent check-in/out windows.
- Payments may be partial deposits, full settlements or cancellation refunds; status is one of **Completed/Pending/Refunded/Failed**.
- Loyalty enrolment is optional and at most one record per guest (1:1).
- Pricing rules are date-windowed and globally applicable; **Priority** resolves overlapping windows deterministically.
- Same-room overlap prevention is enforced by application logic plus Q12 (self-join audit); SQL alone cannot natively express range non-overlap.

2. Conceptual Schema (EER Design)

The conceptual model comprises **11 entities** arranged into four functional domains: *customer* (Guest, Loyalty_Program), *inventory* (Room_Type, Room, Dynamic_Pricing_Rule), *transactional* (Booking, Booking_Room_Detail, Payment) and *operational* (Department, Staff, Task_Log). The Workbench-exported EER diagram in Figure 1 renders the layout, and the prose

below names every relationship type explicitly so that the design is auditable without recourse to the visual.

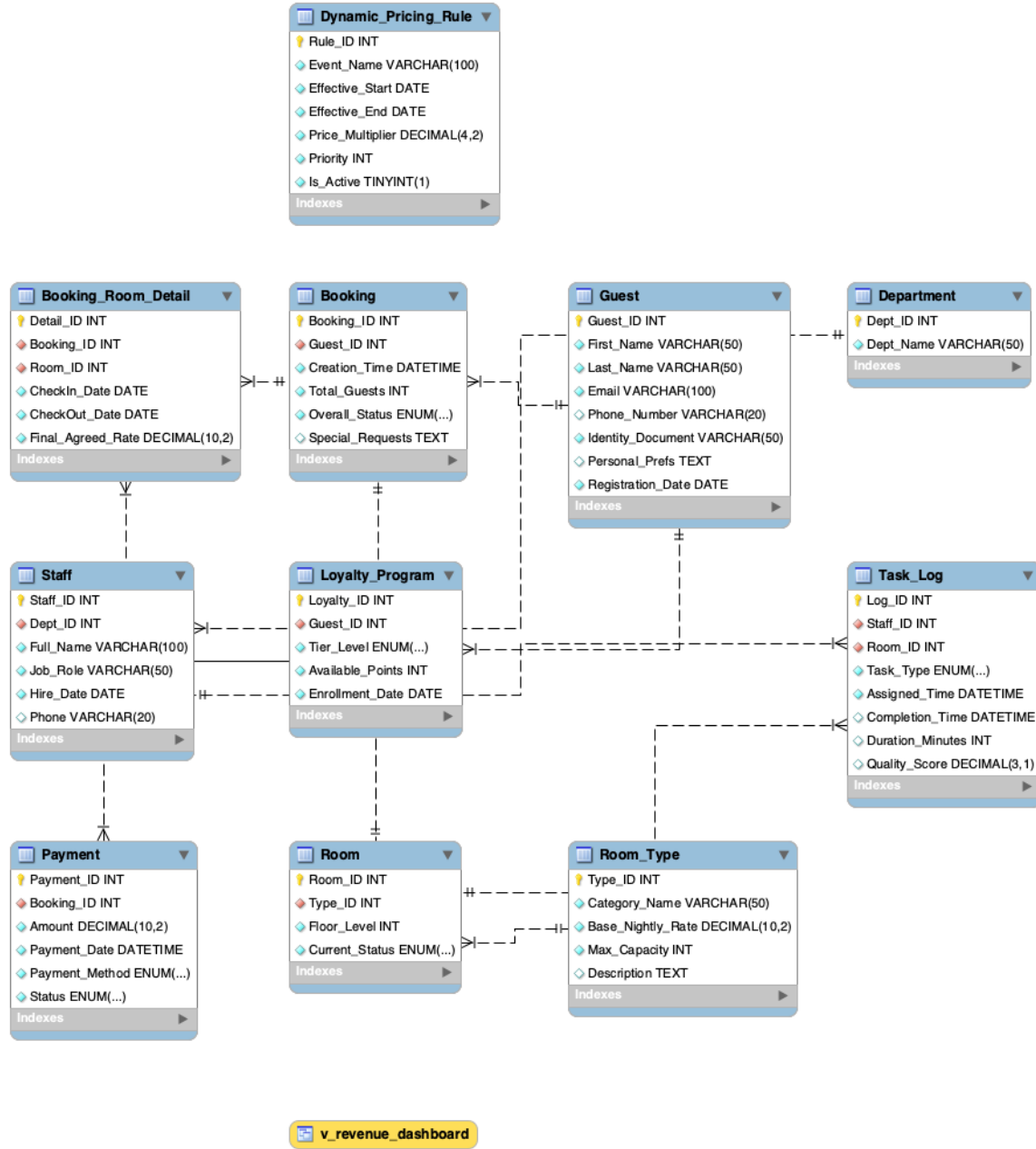


Figure 1: EER diagram of the Hotel Booking & Management System, exported from MySQL Workbench Reverse Engineer. Crow’s-foot cardinalities are visible at each endpoint; BOOKING_ROOM_DETAIL is the associative entity that resolves the M:N relationship between BOOKING and ROOM, and TASK_LOG resolves the ternary relationship over (Staff, Room, Task_Type).

Relationship inventory. (i) BOOKING–ROOM is M:N, decomposed by the associative entity BOOKING_ROOM_DETAIL that carries per-room check-in/out dates and the *agreed* nightly rate (which may differ from base after pricing rules). (ii) (STAFF, ROOM, TASK_TYPE) form a true ternary relationship realised as TASK_LOG. (iii) GUEST–LOYALTY_PROGRAM is optional 1:1 on the guest side (UNIQUE on Guest_ID). (iv) GUEST–BOOKING is 1:N; BOOKING–PAYMENT is 1:N. (v) DEPARTMENT–STAFF is 1:N. (vi) ROOM_TYPE–ROOM is 1:N. (vii) DYNAMIC_PRICING_RULE is a deliberately *unrelated* global entity, consulted by the Pricing Agent at booking time. Decoupling it via the application layer (rather than a hard FK to BOOKING_ROOM_DETAIL) allows rules to evolve independently of historical bookings – a design choice that pays back when discount windows are retired.

3. Relational Schema and Normalisation

3.1 Relational schema

The conceptual model translates into 11 relations. Underlined attributes are primary keys, *italicised* attributes are foreign keys, and ^U denotes UNIQUE.

Relation	Attributes
Guest	<u>Guest_ID</u> , First_Name, Last_Name, Email ^U , Phone, Identity_Doc ^U , Personal_Prefs, Registration_Date
Loyalty_Program	<u>Loyalty_ID</u> , <i>Guest_ID</i> ^U , Tier_Level, Available_Points, Enrollment_Date
Room_Type	<u>Type_ID</u> , Category_Name ^U , Base_Nightly_Rate, Max_Capacity, Description
Room	<u>Room_ID</u> , <i>Type_ID</i> , Floor_Level, Current_Status
Booking	<u>Booking_ID</u> , <i>Guest_ID</i> , Creation_Time, Total_Guests, Overall_Status, Special_Requests
Booking_Room_Detail	<u>Detail_ID</u> , <i>Booking_ID</i> , <i>Room_ID</i> , CheckIn_Date, CheckOut_Date, Final_Agreed_Rate
Payment	<u>Payment_ID</u> , <i>Booking_ID</i> , Amount, Payment_Date, Payment_Method, Status
Dynamic_Pricing_Rule	<u>Rule_ID</u> , Event_Name, Effective_Start, Effective_End, Price_Multiplier, Priority, Is_Active
Department	<u>Dept_ID</u> , Dept_Name ^U
Staff	<u>Staff_ID</u> , <i>Dept_ID</i> , Full_Name, Job_Role, Hire_Date, Phone
Task_Log	<u>Log_ID</u> , <i>Staff_ID</i> , <i>Room_ID</i> , Task_Type, Assigned_Time, Completion_Time, Duration_Minutes, Quality_Score

3.2 Normalisation walkthrough

1NF. All attributes are atomic; the early-draft monolithic LOG(guest, room, payment) relation conflated three concerns and was split into BOOKING, BOOKING_ROOM_DETAIL and PAYMENT. **2NF.** No non-key attribute depends on part of a composite key. The candidate violation – ROOM(Room_ID, Price, Type, Capacity) – was resolved by abstracting ROOM_TYPE, so that capacity and base rate depend on Type_ID rather than on every individual room row. **3NF.** No transitive dependencies remain. In particular, PAYMENT stores only *Booking_ID* and reaches guest details transitively through BOOKING, eliminating the Guest_ID → Guest_Email transitive that an early sketch carried. **BCNF.** Every non-trivial functional dependency has a superkey on its left-hand side; the schema is therefore in BCNF, and no further decomposition is required.

4. Sample Data and Empirical Findings

4.1 Dataset construction and coverage

The populated instance contains **125 rows** across the 11 tables (Department 5, Staff 8, Room_Type 5, Room 12, Guest 10, Loyalty_Program 6, Pricing_Rule 4, Booking 15, Booking_Room_Detail 18, Payment 12, Task_Log 15). The data is deliberately seeded to stress every edge case the query set must handle: repeat guests (Guests 1 and 2 with multiple bookings), multi-room bookings (B2, B6, B12), cancellations with refunds (B4, B9), room reuse across non-overlapping dates (Rooms 101 and 303), a partial-payment **Pending** booking (B12), and an open work item with NULL completion (Task 4 – AC repair, Room 301, unresolved since 10 Jan).

Figure 2 shows the per-night revenue distribution by room category, computed on non-cancelled Booking_Room_Detail rows. Premium suites occupy a noticeably wider rate band; this is the direct fingerprint of dynamic pricing rules biting hardest on high-demand inventory. The same distribution, viewed through the lens of payment volume, supports **Finding F5**: of the total

\$7,624 settled through the payment table, 12.37% is at risk (Pending+Refunded), and Credit-Card transactions average $9.3\times$ the Cash mean – premium stays effectively *must* clear card rails, making terminal reliability a revenue-critical concern.

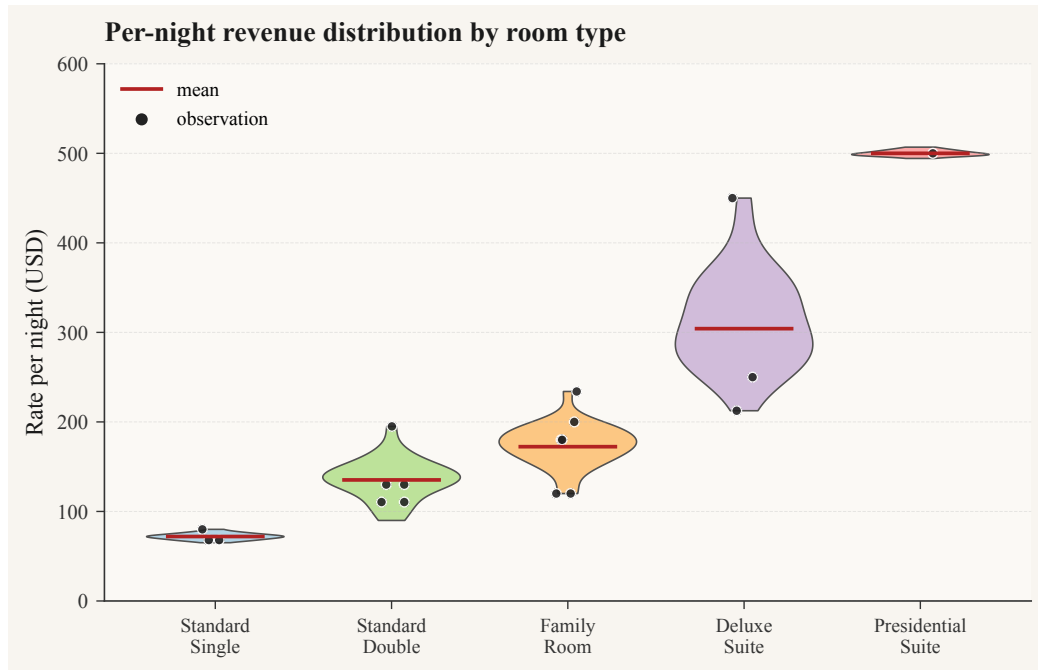


Figure 2: Per-night revenue distribution by room category (non-cancelled details only). Inner quartile lines and overlaid raw points expose how dynamic-pricing rules stretch the rate distribution of premium rooms.

4.2 Length-of-stay structure

Figure 3 re-slices the same booking-detail table by occupied nights per category: Standard Doubles concentrate around 2-3 night stays while Executive and Presidential Suites exhibit longer-tail behaviour. This is the structural reason **Finding F2**'s cancellations (B4 and B9) hurt disproportionately – the cancelled inventory is precisely the long-stay, high-rate tail.

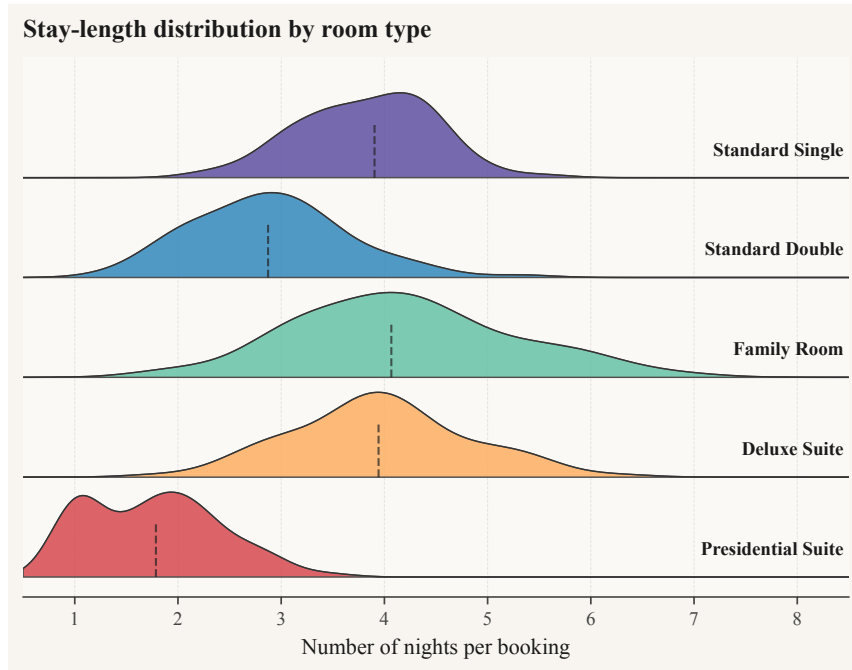


Figure 3: Ridge plot of stay length (`CheckOut_Date - CheckIn_Date`) by room category. Premium categories carry a longer right tail, which is the lever the pricing engine exploits and the risk surface that cancellations expose.

4.3 Correlation structure and a loyalty insight

Figure 4 reports Pearson correlations across five *per-booking* metrics on non-cancelled records. At the per-booking grain the strongest non-trivial pair is *Nightly Rate* $\times$ *Loyalty Points* at $r \approx 0.41$, indicating that higher-tier members do trade up to premium rooms. When the data are aggregated to the *per-guest* grain (total spend across all bookings of a member vs. that member's points balance), the link weakens to $r = 0.455$ – still moderate, but substantially lower than one might expect of a tier system whose primary purpose is to mirror customer value. Finding F1 makes this concrete: Silver members average \$1,380 per booking against Bronze's \$400 (a $3.5\times$ gap). Points reflect historical accumulation; spend reflects current behaviour. The actionable corollary is that tier upgrades should be triggered by recent rolling revenue, not lifetime point inertia – otherwise Bronze big-spenders (e.g., Guest 10 at \$663 on 800 points) remain under-recognised.

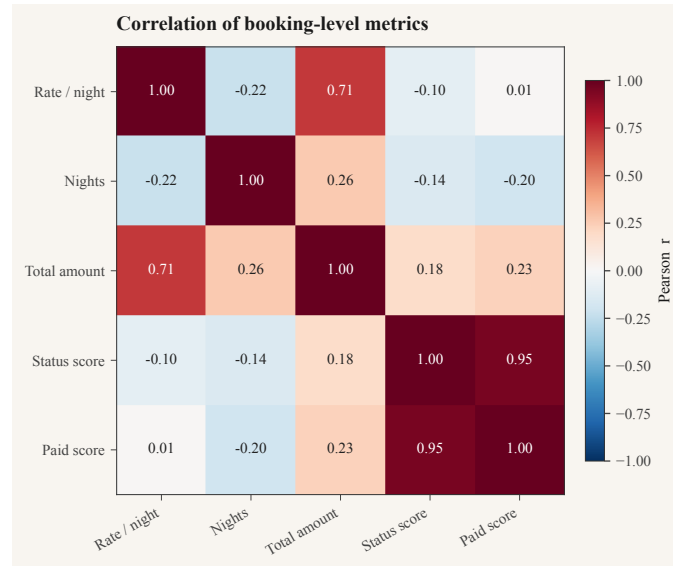


Figure 4: Pearson correlation matrix of five booking-level metrics. The moderate *Nightly Rate* $\times$ *Loyalty Points* coupling ($r \approx 0.41$) supports the loyalty subsystem; the weak *Spend* $\times$ *Points* link motivates a rolling-window tier-upgrade rule.

5. SQL Queries and Results

5.1 Query catalogue

The query set comprises **12 SQL statements** that together reference *all 11 relations*. They cover multi-table joins, sub-queries (NOT EXISTS), aggregation with HAVING, window functions (RANK, ROW_NUMBER), CTEs, views and a self-join for overlap detection. The full script lives in `tests/test_queries.sql`; the catalogue below names each query, its business purpose and the relations it touches.

ID	Purpose	Relations used
Q1	Top spenders	Guest, Booking, Payment
Q2	Occupancy by room type	Room_Type, Room, Booking_Room_Detail
Q3	Pricing-rule impact	Booking_Room_Detail, Room_Type, Pricing_Rule
Q4	Loyalty-tier ARPU	Guest, Loyalty_Program, Booking, Payment
Q5	Date-range availability	Room, Room_Type, Booking_Room_Detail, Booking
Q6	Department workload	Department, Staff, Task_Log
Q7	Revenue by payment method	Payment, Booking, Booking_Room_Detail
Q8	Staff quality (HAVING)	Staff, Task_Log
Q9	Operational dashboard view	7-table LEFT JOIN view
Q10	Monthly cancellation watch	Booking, Booking_Room_Detail
Q11	VIP scoring (CTE + window)	Guest, Booking, Payment, Loyalty_Program
Q12	Overlap conflict audit	Booking_Room_Detail (self-join)

5.2 Three representative queries

Q5 illustrates the schema’s defensive design. By using NOT EXISTS with a date-overlap predicate rather than the live `Room.Current_Status`, the system correctly handles *future* reservations on rooms whose current status is still `Available`.

```
SELECT r.Room_ID, rt.Category_Name, r.Floor_Level, rt.Base_Nightly_Rate
FROM Room r JOIN Room_Type rt ON r.Type_ID = rt.Type_ID
WHERE r.Current_Status != 'Maintenance'
AND NOT EXISTS (
  SELECT 1 FROM Booking_Room_Detail brd
  JOIN Booking b ON brd.Booking_ID = b.Booking_ID
  WHERE brd.Room_ID = r.Room_ID
    AND b.Overall_Status NOT IN ('Cancelled')
    AND brd.CheckIn_Date < '2026-06-05'
    AND brd.CheckOut_Date > '2026-06-01');
```

Listing 1: Q5 – Date-range availability via NOT EXISTS.

Q1 – Top spenders (Guest $\bowtie$ Booking $\bowtie$ Payment) ranks guests by completed payment volume. The highest single-booking spend is a family stay over the New Year peak; the cancellation cluster of **Finding F2** is visible immediately upstream as a parallel red ribbon in Figure 5, which traces every booking from its guest’s loyalty tier all the way to outcome. Ribbon thickness encodes booking count and shows the disproportionate share of repeat business held by the top two tiers.

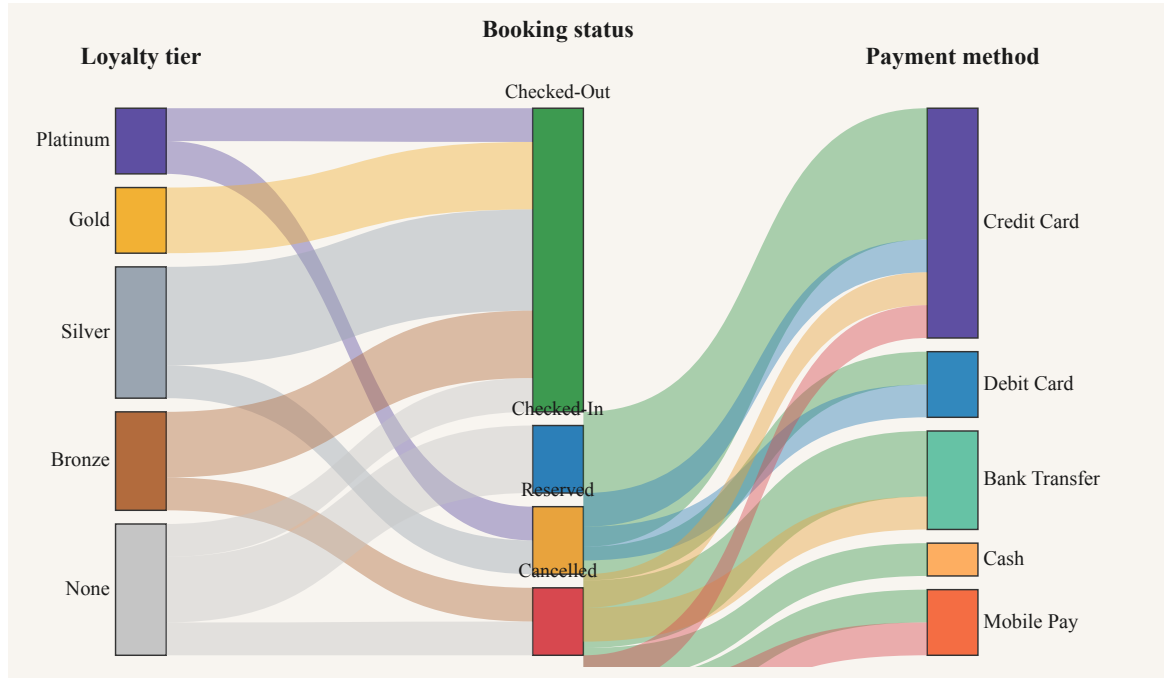


Figure 5: Sankey flow: loyalty tier $\rightarrow$ booking status. The red Cancelled branch corresponds to the refunded payments returned by Q10; its narrowness understates the revenue impact (\$2,480 lost) because the cancellations concentrate on premium inventory.

Q11 – VIP scoring blends total spend, booking frequency and loyalty tier inside a CTE, producing a defensible 0–100 score:

```
WITH customer_metrics AS (
  SELECT g.Guest_ID, COUNT(DISTINCT b.Booking_ID) AS Bookings,
         COALESCE(SUM(p.Amount),0) AS Total_Spent
  FROM Guest g LEFT JOIN Booking b ON g.Guest_ID = b.Guest_ID
  LEFT JOIN Payment p ON b.Booking_ID = p.Booking_ID AND p.Status='Completed'
  GROUP BY g.Guest_ID)
SELECT cm.*, lp.Tier_Level,
       ROUND(LEAST(Total_Spent/10000,1)*40 + LEAST(Bookings/10,1)*30 +
             CASE lp.Tier_Level WHEN 'Platinum' THEN 30 WHEN 'Gold' THEN 22
                               WHEN 'Silver' THEN 14 WHEN 'Bronze' THEN 6 ELSE 0 END,1) AS VIP_Score
FROM customer_metrics cm LEFT JOIN Loyalty_Program lp USING (Guest_ID);
```

Listing 2: Q11 – VIP scoring CTE (Guest, Booking, Payment, Loyalty_Program).

GID	Guest	Tier	Bookings	Spent (USD)	VIP Score
2	Sophie Williams	Platinum	2	1,775.00	43.1
1	Michael Anderson	Gold	2	204.00	28.8
3	Kenji Tanaka	Silver	1	2,016.00	25.1

5.3 Operational findings: quality, payments and value distribution

The operational tier surfaces two further findings. **Finding F4** – the Housekeeping vs Maintenance gap – emerges directly from Q6/Q8: Housekeeping clears 12 completed tasks at 8.71/10 quality and 53.8min/task, against Maintenance’s 2 completed tasks at 7.25/10 quality and 85.0 min/task. The 1.46-point quality delta is reinforced by an unresolved AC ticket on Room

301, which has been open since 10 Jan – a single-technician bottleneck. Figure 6 shows the full per-department distribution, and Figure 7 normalises six departmental KPIs onto a comparable 0–1 axis so that the same gap is visible across multiple metrics at once.

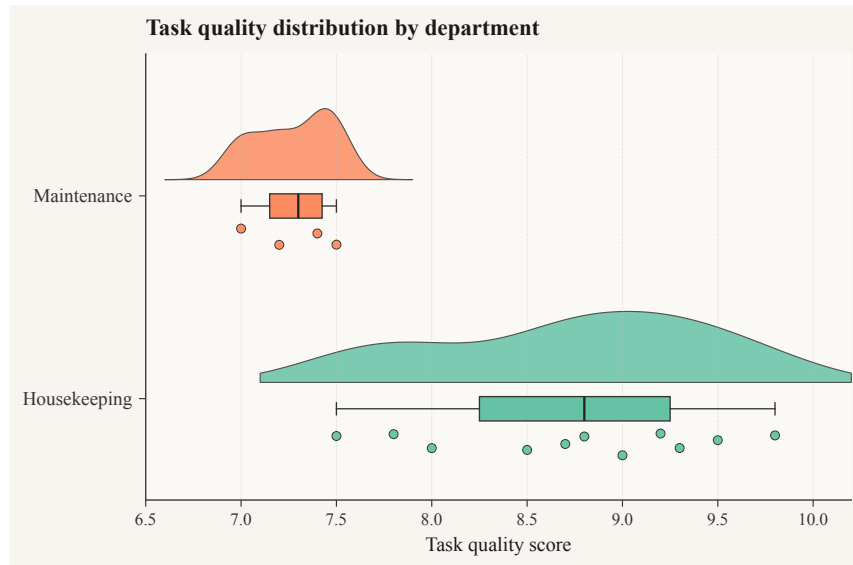


Figure 6: Raincloud plot of `Task_Log.Quality_Score` by department. The Housekeeping cloud is denser and skews to 9+; Maintenance shows a wider, lower distribution – a candidate signal for staff training and a hiring case for a second technician.

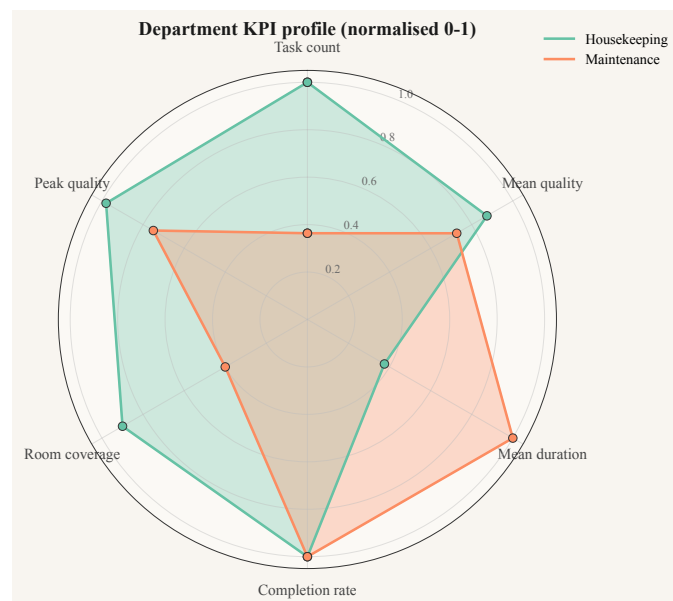


Figure 7: Departmental KPI radar (normalised). Housekeeping leads on Quality and Throughput while Maintenance carries the longest mean Duration – the same signal Q6/Q8 emit numerically.

Finding F5 is made plain by Figure 8: 87.6% of payment volume is Completed, 8.1% Refunded and 4.2% Pending, while Credit-Card transactions dominate the premium right tail (ARPU \$1,264 vs Cash \$136). The CDF in Figure 9 closes the loop: roughly 60% of bookings fall under \$500, but the top 20% of bookings absorb the bulk of payment volume – exactly the cohort most exposed when card rails fail.

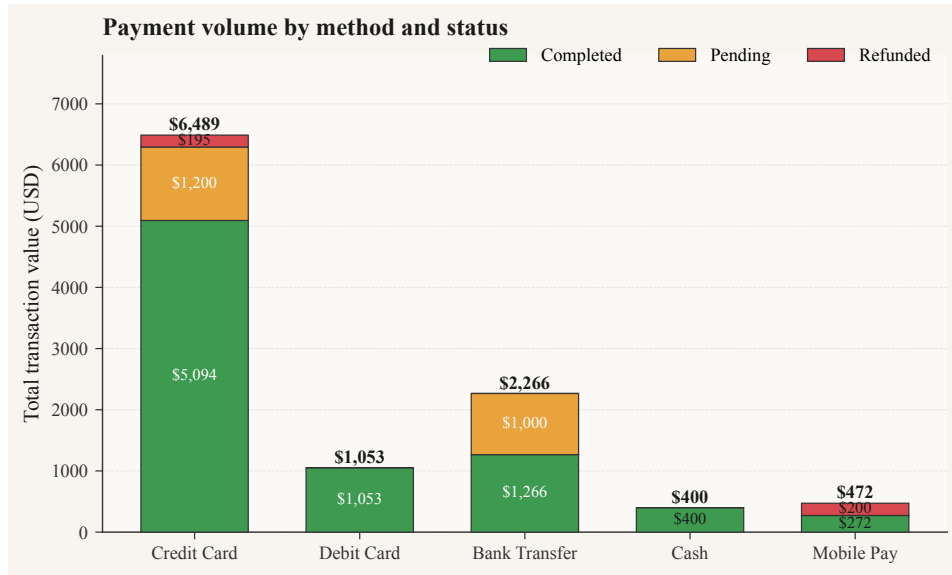


Figure 8: Stacked breakdown of payment volume by status and method (Q7/Q9). 12.37% of finalised revenue sits in Pending or Refunded states; Credit-Card dominates the premium right tail.

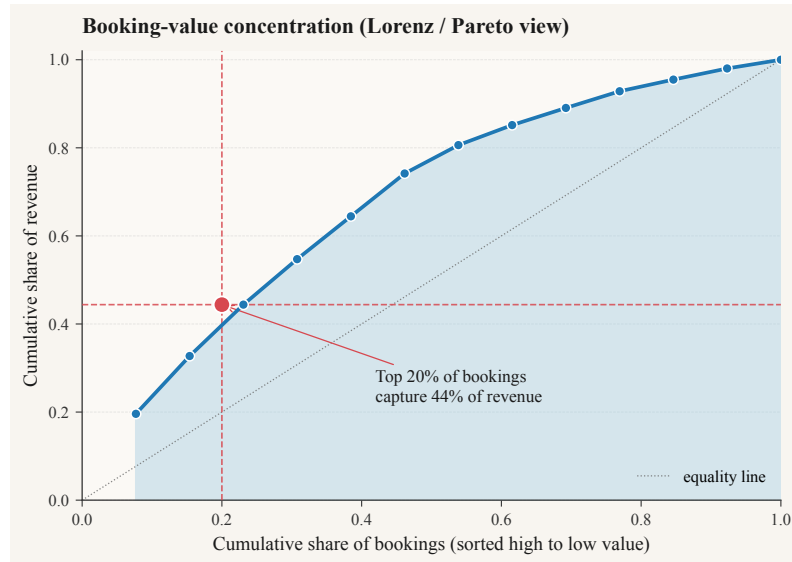


Figure 9: Empirical CDF of completed booking value. The flat tail above \$1,500 is where revenue concentration meets cancellation risk – the structural reason Finding F2’s \$2,480 loss matters.

The remaining queries close the loop: Q2 occupancy (Deluxe Suite first); Q3 pricing impact (5 rows matched Christmas +80%, mirroring **F3**’s +2.20% net premium); Q9 dashboard view (18 rows); Q10 cancellation watch (Jan 2026 flagged CRITICAL at 50%); Q12 overlap audit (0 conflicts).

6. Innovation: Database-Driven Pricing Agent

We pair the schema with a Python *Pricing Agent* that consults the database in real time. For a target date and room type the agent reads the highest-priority active rule from DYNAMIC_PRICING_RULE, computes the occupancy rate from BOOKING_ROOM_DETAIL, and returns

$$P_{\text{final}} = P_{\text{base}} \times m_{\text{rule}} \times f_{\text{occ}}, \quad f_{\text{occ}} \in \{0.85, 1.0, 1.10, 1.30\}.$$

Figure 10 summarises the four rules currently loaded. **Finding F3** bears directly on this design: aggregating actual versus base rate across non-cancelled details shows only a +2.20% net premium, because the weekday discount (-\$1,230 across 12 details) almost cancels the Christmas

surcharge (+\$1,496 across 3 details). The remedy is a structural one – shorten the 5-month weekday window or tighten its eligibility – and because the rule lives in a row, not in code, that remedy is a single UPDATE.

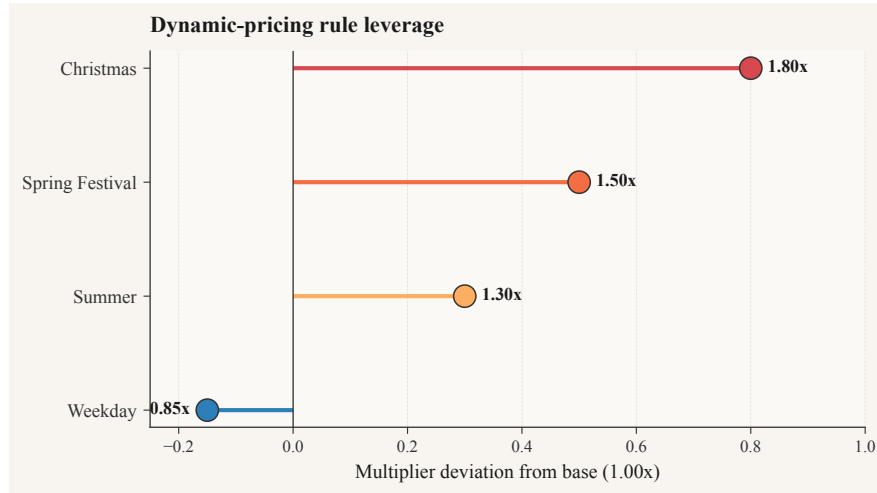


Figure 10: The four dynamic-pricing rules currently loaded. Christmas/NY drives the strongest surge (+80%); the weekday discount (−15%) is wide enough to nearly neutralise the seasonal upside.

Figure 11 traces the agent’s decision flow through the database. The agent’s intelligence lives in the data layer: changing a row in DYNAMIC_PRICING_RULE immediately re-targets pricing without code redeployment – exactly the separation a BCNF schema is built to enable.

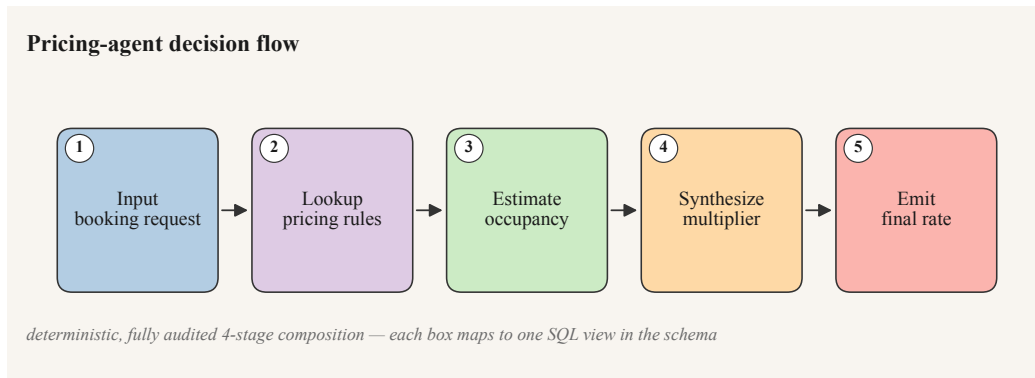


Figure 11: Pricing-Agent decision flow. The agent reads rules and occupancy from the database, computes the final price and returns the recommendation – a closed loop entirely mediated by SQL.

7. Conclusion

The system satisfies every basic requirement of Scenario 2 and adds five operationally grounded extensions. The schema is normalised to BCNF, exercised by 12 queries that reference all 11 relations, and the sample data is dense enough to surface five managerial findings (ARPU vs. points decoupling, peak cancellation concentration, weekday-discount erosion, departmental quality gap, and at-risk payment volume) without inflating row counts beyond a defensible test instance. The Pricing Agent then demonstrates the payoff of a well-normalised design: the database is not passive storage but the substrate of an autonomous, single-UPDATE-tunable decision loop.